HELWAN NATIONAL UNIVERSITY Faculty of Engineering

Department of Robotics & Mechatronics

Subject: Electro-Mechanical Machines

Course Code: POW2305

Title: Closed loop Angle Grinder

Students Names:	*Student ID:*
Youssef Ahmed Rady Elbeltagy	*922230111*
Mohamad Sherif Shabrawy	*922230084*

Submitted TO: Dr. Abdullah Mohamad
Teaching Assistant: Ahmad Abdelal

Design and Implementation of an IoT-Enabled Closed-Loop Speed Control System for a DC Motor Using ESP32

Abstract

This project presents the design and implementation of an IoT-enabled closed-loop speed control system for a high-speed brushed DC motor using an ESP32 microcontroller. The system integrates encoder-based speed feedback, a feed-forward duty-cycle model, and a PID controller to achieve accurate speed regulation over a wide operating range (0–12,000 RPM). User interaction is supported through a keypad, potentiometer, local LCD display, and a cloud-based Blynk interface, enabling both local and remote control. Safety mechanisms including stall detection, overspeed protection, soft start/stop routines, and direction change handling are incorporated to ensure reliable operation. Experimental results demonstrate stable speed tracking, zero steady-state error under load, and robust performance under dynamic setpoint changes.

1. Introduction

DC motors are widely used in industrial and embedded applications due to their simplicity, high torque-to-size ratio, and ease of speed control. However, achieving accurate speed regulation over a large operating range requires closed-loop control techniques that compensate for nonlinearities, load variations, and supply fluctuations.

This project focuses on the development of a smart motor control system that combines classical control theory with modern IoT capabilities. The system is designed to:

- Precisely regulate motor speed using encoder feedback
- Allow both local and remote control of speed and direction
- Provide real-time monitoring and safety protection
- Demonstrate practical application of PID control in embedded systems

2. System Architecture

2.1 Hardware Components

The system is composed of the following main hardware components:

- **775 Brushed DC Motor (12V)**
  Capable of operating between 7,000 and 12,000 RPM with high torque output.

- **Cytron MD13S Motor Driver**
  Provides PWM-based speed control and direction control with up to 13 A continuous current capability. Integrated over-current and thermal protection enhance system safety.
- **ESP32 DevKit Microcontroller**
  Acts as the main controller, providing PWM generation, encoder interfacing, ADC readings, and Wi-Fi connectivity.
- **Rotary Encoder**
  Provides quadrature pulse feedback for real-time speed measurement.
- **Buck Converter**
  Steps down the 12 V supply to a regulated 5 V level for powering the ESP32.
- **User Interface Components**
  - Keypad for numeric speed entry and direction selection
  - Potentiometer for analog speed control
  - I2C LCD for local feedback
- **Power Supply (12 V, 15 A)**
  Supplies sufficient current for both motor operation and control electronics.

2.2 Software Architecture

The firmware is structured into modular subsystems:

- Input handling (keypad, potentiometer, Blynk)
- Speed measurement and filtering
- Feed-forward duty prediction
- PID control loop
- Safety monitoring
- Display and IoT synchronization

A periodic control loop is executed using a timer interrupt at 150 ms intervals.

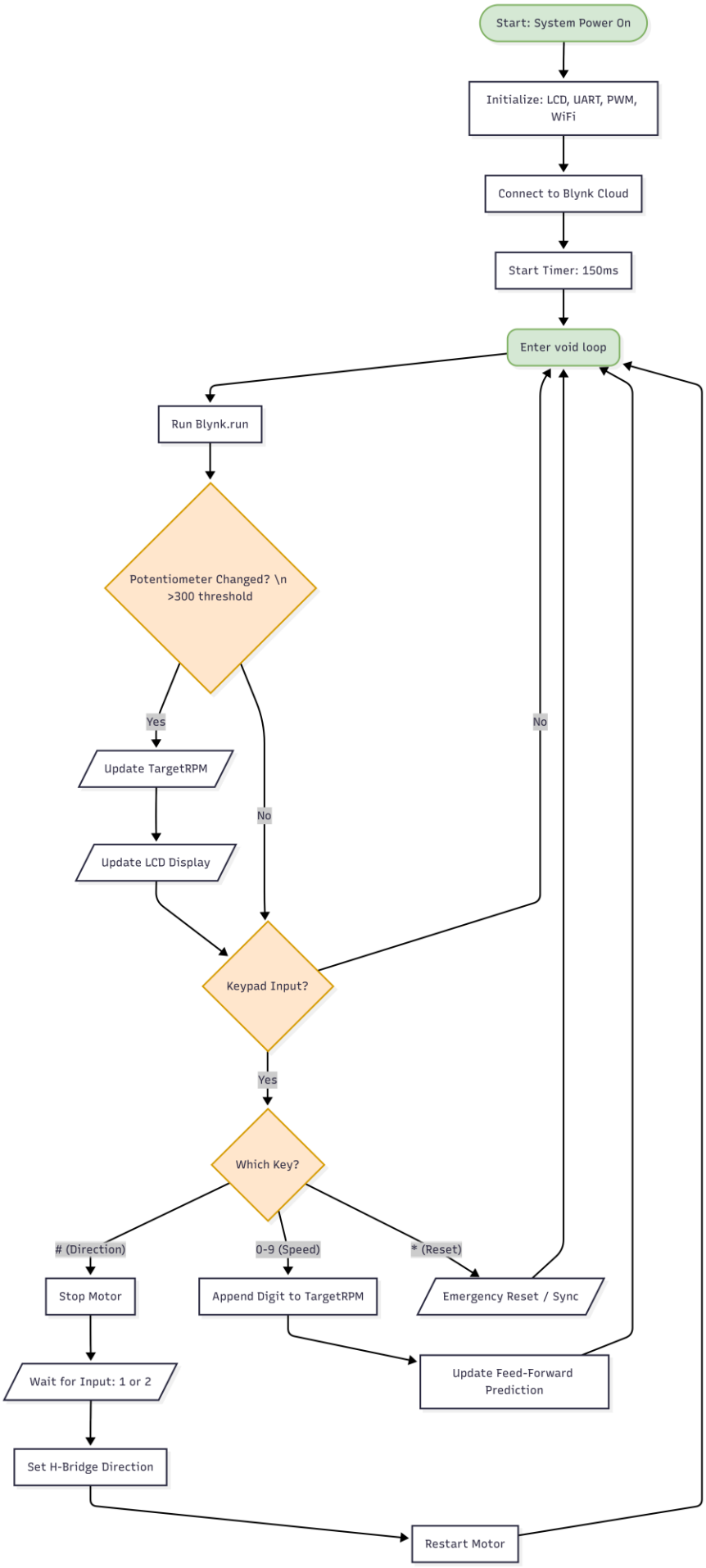
Start: System Power On
Initialize: LCD, UART, PWM, WiFi
Connect to Blynk Cloud
Start Timer: 150ms
Enter void loop
Run Blynk.run
Potentiometer Changed? \n >300 threshold
Yes
No
Update TargetRPM
Update LCD Display
Keypad Input?
No
Yes
Which Key?
(Direction)
0-9 (Speed)
* (Reset)
Stop Motor
Append Digit to TargetRPM
Emergency Reset / Sync
Update Feed-Forward Prediction
Wait for Input: 1 or 2
Set H-Bridge Direction
Restart Motor

Figure 2.2: Flowchart of the Main System Loop, illustrating the non-blocking architecture handling WiFi synchronization, user inputs, and mode switching.

2.3 Electrical Interface and Pin Configuration

To ensure hardware abstraction and modularity, the ESP32 GPIOs were configured with specific electrical characteristics. The system operates on a logic level of 3.3V, interfacing with the 12V motor subsystem via the motor driver.

- **PWM Generation:** The control signal is generated on **GPIO 23** using the ESP32's LEDC peripheral. The PWM frequency is set to **20 kHz** to minimize audible motor whine, with a resolution of **10 bits** (0–1023 duty cycle steps), allowing for fine-grained voltage control.
- **Quadrature Encoder:** The encoder channels A and B are connected to **GPIO 34 and GPIO 35** respectively. These pins are configured as `INPUT_PULLUP` and utilize hardware interrupts to capture state changes on both rising and falling edges, maximizing measurement resolution.
- **User Interface:**
    - **Keypad:** A 4x3 matrix keypad is mapped to GPIOs 19, 18, 5, 17 (Rows) and 2, 16, 4 (Columns).
    - **Potentiometer:** Analog speed reference is read via **GPIO 36 (VP)** using the ESP32's ADC.
    - **I2C LCD:** Driven via standard SDA/SCL lines using the `LiquidCrystal_I2C` library at address 0x27.

3. Speed Measurement and Signal Processing

Motor speed is measured using a quadrature encoder with a physical resolution of 7 pulses per revolution (PPR). By counting both rising and falling edges on channels A and B, the effective resolution is quadrupled to **28 counts per revolution (CPR)**.

The control loop operates at a sampling interval (Ts) of **150 ms**. Instantaneous speed is calculated based on the pulse count accumulated over this interval.

$$\text{RPM} = \frac{\text{Pulse Count}}{\text{CPR}} \times \frac{60}{\Delta t}$$

To mitigate quantization noise and measurement jitter, a moving average filter is applied. The firmware utilizes a circular buffer of size N=12, calculating the smoothed RPM as:

$$RPM_{smooth} = \frac{1}{12} \sum_{i=0}^{11} RPM_{raw}[i]$$

This specific buffer size was selected to balance signal smoothness against phase lag, ensuring the controller remains responsive to rapid setpoint changes.

4. Advanced Feed-Forward Control with Adaptive Bias

Due to the nonlinear relationship between PWM duty cycle and motor speed, a feed-forward duty-cycle map was experimentally derived. The map associate's discrete RPM values with corresponding PWM duty values.

To linearize the plant model, a lookup table (`DutyMap`) containing approximately 40 distinct RPM-to-Duty operating points (ranging from 0 to 12,284 RPM) was experimentally derived. The system employs **Linear Interpolation** to calculate the base duty cycle (D base) for any requested target speed between these points.

$$D_{\text{base}} = D_0 + \alpha(D_1 - D_0)$$

where:

- α is the interpolation ratio between adjacent speed points.

4.1 Adaptive Bias Correction

A novel feature of this implementation is the **Adaptive Feed-Forward Bias**. Since the static lookup table cannot account for battery voltage drop or varying mechanical loads over time, the system implements an online learning algorithm. When the steady-state error persists between 10 and 500 RPM, the system dynamically adjusts a bias term (`dutyBias`) for the specific segment of the lookup table:

$$Bias_{new} = Bias_{old} + \gamma \times (DutyPerRPM) \times Error$$

Where gamma is a learning rate of 0.02. This allows the controller to "learn" the changing system characteristics in real-time, effectively shifting the feed-forward curve to match current operating conditions.

5. PID Control Design

To eliminate steady-state errors and improve transient response, a PID controller is implemented on top of the feed-forward term.

The control law is given by:

$$u(t) = D_{\text{base}} + K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt}$$

where:

- $e(t) = \text{RPM}_{\text{target}} - \text{RPM}_{\text{measured}}$

The feedback mechanism utilizes a discrete PID algorithm with the following parameters tuned for the 775 DC motor:

- $K_p = 0.02$
- $K_i = 0.00015$
- $K_d = 0.0006$

5.1 Noise Suppression and Stability

To prevent high-frequency noise from amplifying through the derivative term, a **Digital Low-Pass Filter (Exponential Moving Average)** is applied to the derivative calculation with a smoothing factor of alpha = 0.62:

$$D_{filtered} = 0.62 \cdot D_{prev} + (1 - 0.62) \cdot D_{raw}$$

5.2 Integral Windup and Deadband

To ensure system stability:

1. **Integral Windup Protection:** The integral term is strictly constrained between -12,000 and +12,000 to prevent integrator saturation during large error transients.

2. **Deadband Zone:** A deadband of + or - 5 RPM is implemented. If the error falls within this range, it is treated as zero, preventing control of output oscillation (chattering) around the setpoint.

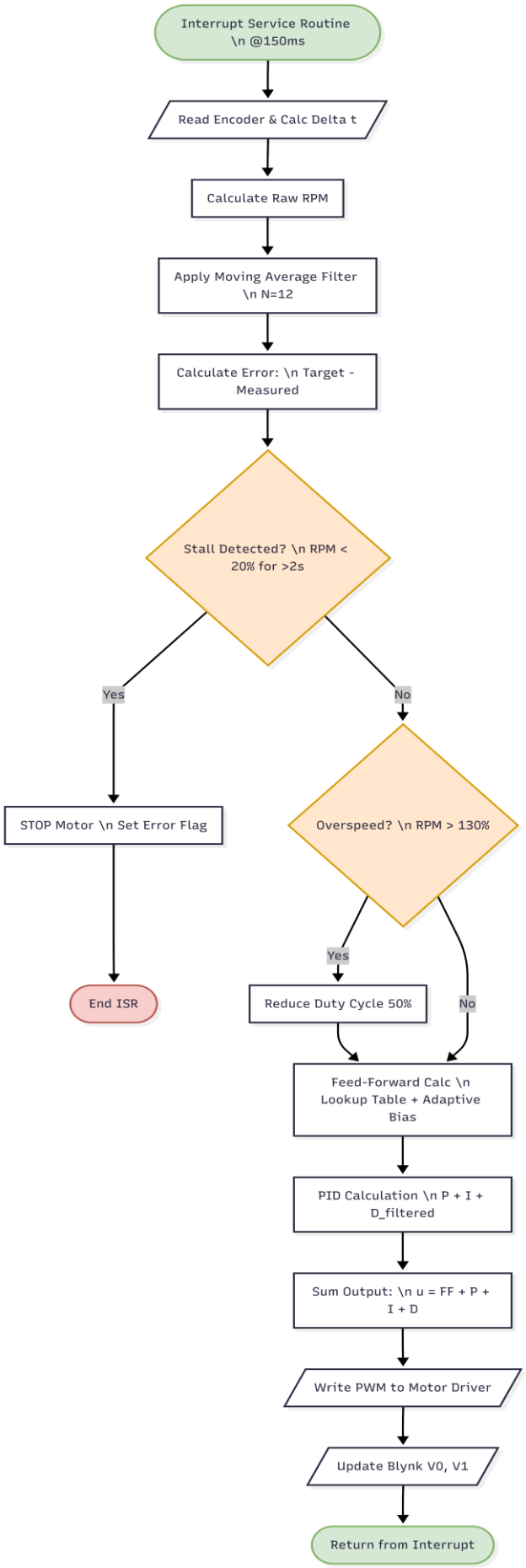
Interrupt Service Routine \n @150ms
Read Encoder & Calc Delta t
Calculate Raw RPM
Apply Moving Average Filter \n N=12
Calculate Error: \n Target - Measured
Stall Detected? \n RPM < 20% for >2s
Yes
No
STOP Motor \n Set Error Flag
End ISR
Overspeed? \n RPM > 130%
Yes
No
Reduce Duty Cycle 50%
Feed-Forward Calc \n Lookup Table + Adaptive Bias
PID Calculation \n P + I + D_filtered
Sum Output: \n u = FF + P + I + D
Write PWM to Motor Driver
Update Blynk V0, V1
Return from Interrupt

Figure 5.2: Logic flow of the Periodic Interrupt Service Routine (ISR) executing every 150ms. The routine prioritizes safety checks (Stall/Overspeed) before executing the adaptive Feed-Forward and PID control algorithms.

6. Safety and Protection Mechanisms

Several protection strategies are implemented:

- **Stall Detection:** The system monitors if the `liveRPM` falls below 20% of `targetRPM`. If this condition persists for **2000 ms** (2 seconds) while the target is above 200 RPM, the motor is emergency stopped, and a "STALL ERROR" is triggered on the local display.

- **Overspeed Protection:** If `liveRPM` exceeds 130% of `targetRPM` for more than **1000 ms**, the controller automatically halves the duty cycle to protect the mechanical assembly.
- **Soft Start / Soft Stop:** To reduce inrush current, changes in duty cycle are ramped. The ramp step size is dynamic, calculated as (Delta Duty / 30, 2), ensuring a smooth transition regardless of the speed jump magnitude.

- **Safe Direction Change**
  Direction changes are only allowed after the motor has been brought to a complete stop.

7. IoT and Human-Machine Interface (HMI)

The system operates a dual-interface architecture allowing synchronized control via physical hardware and the Blynk IoT Cloud.

7.1 Blynk Virtual Pin Mapping

- **V0:** Live RPM Telemetry (Output to Graph).
- **V1:** Target RPM Setpoint (Input/Output sync).
- **V2:** Direction Control (0 = Forward, 1 = Reverse).
- **V3:** Master Start/Stop Switch.
- **V4:** Remote LCD Widget (Mirrors physical LCD).

7.2 Input Arbitrator

The firmware acts as an arbitrator between inputs. The physical keypad (**#** key) allows the user to interrupt operation to change direction. The potentiometer input utilizes a **15-sample moving average** to filter electrical noise; control authority is granted to the potentiometer only when a change in value exceeds a hysteresis threshold (300 raw ADC counts), preventing drift from overriding IoT commands.

8. Experimental Results and Discussion

Experimental testing demonstrated:

- Accurate tracking of speed setpoints across the full operating range
- Zero steady-state error under constant load conditions
- Stable operation during rapid setpoint changes
- Effective suppression of noise through filtering and feed-forward compensation

The combination of feed-forward control and PID feedback significantly improved response time compared to a pure PID approach.

9. Conclusion

This project successfully demonstrates a robust, IoT-enabled DC motor speed control system that integrates classical control theory with modern embedded and cloud technologies. The use of feed-forward modeling, adaptive bias correction, and PID control results in high accuracy, stability, and safety. The system architecture is scalable and can be extended to higher-power motors, additional sensors, or edge-based AI monitoring in future work.

10. Future Work

To elevate this system from a prototype to an industrial-grade solution, the following technical extensions are proposed:

- **Model-Based Control & System Identification:** Future iterations will implement mathematical modeling to derive the motor's transfer function parameters (inertia, friction, back-EMF). This enables advanced control strategies like **Linear Quadratic Regulation (LQR)** or **Model Predictive Control (MPC)** to optimize performance beyond standard PID limitations.
- **Edge-AI Processing on Single Board Computers:** Migrating high-level processing to a **Raspberry Pi** would allow for **Neural Network** deployment. This enables features such as real-time PID auto tuning and intelligent load classification, while the ESP32 remains dedicated to real-time driver actuation.
- **Industrial Communication Protocols:** To ensure reliability in electrically noisy environments, the current Wi-Fi interface should be supplemented with **RS-485 (Modbus)** or **CAN Bus**. These differentials signaling protocols provide superior immunity to Electromagnetic Interference (EMI) and allow direct integration with industrial PLCs.
- **Predictive Maintenance via Signal Analysis:** By integrating current sensors and accelerometers, the system could perform **Fast Fourier Transform (FFT)** analysis on vibration and current signatures. This allows for the early detection of mechanical anomalies—such as bearing wear or misalignment—before catastrophic failure occurs.

11. Appendix

11.1 Firmware Source Code

The complete firmware source code for the ESP32 controller is hosted in a digital repository. This includes the implementation of the PID algorithm, the feed-forward linearization map, and the Blynk IoT integration logic.

Access Instructions: Scan the QR code below using a smartphone camera to view the full C source file, or visit the direct link provided.

Figure B.1: QR Code linking to the `Closed_Loop_Motor_Control.cpp` source file.

Direct Repository Link: [Code is here]